ReLog: Execution-Aware Logging with Runtime Feedback for LLM-Oriented Debugging

Xin Wang*
xwang496@connect.hkust-gz.edu.cn
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, China

Yang Feng*
1921567337@qq.com
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, China

Xiaoqian Jiao
xjiao393@connect.hkust-gz.edu.cn
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, China

Yang Zhang
uzhangyang@foxmail.com
Hebei University of Science and Technology
Hebei, China

Zhenhao Li
lzhenhao@yorku.ca
York University
Toronto, Canada

Zishuo Ding†
zishuoding@hkust-gz.edu.cn
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, China

arXiv:2603.29122v3 [cs.SE] 26 Aug 2026

Abstract

Logging statements are important for software debugging and maintenance. However, existing approaches to automatic logging statement generation primarily rely on static code analysis, generating statements in a single pass without considering actual program execution. Moreover, they typically evaluate generated statements by comparing them against developer-written ones, implicitly assuming that the original statements constitute an adequate gold standard. This assumption is increasingly restrictive in the LLM era: logging statements are consumed not only by human developers but also by LLMs for downstream tasks. As a result, generating logging statements solely for human developers, and evaluating them solely by similarity to developer-written statements, does not necessarily reflect their practical usefulness.

To overcome these limitations, we introduce **ReLog**, an iterative logging generation framework guided by continuous runtime feedback. **ReLog** leverages LLMs to generate, execute, evaluate, and iteratively refine logging statements based on runtime feedback, so that the runtime logs provide sufficient information for downstream tasks. Instead of measuring similarity to developer-written statements, we evaluate **ReLog** through downstream tasks (i.e., defect localization and repair). We construct a new benchmark derived from Defects4J under two settings, namely direct and indirect debugging. The results show that **ReLog** consistently outperforms all baselines across both settings. In particular, it achieves the highest bug-detection F1 score of 0.520 and successfully repairs 97 defects in the direct setting, while also obtaining the best F1 score of 0.408 in the indirect setting where source code is unavailable. Additional experiments across upstream generation models and downstream log-consuming models demonstrate the generality of the framework, while ablation studies confirm the importance of iterative refinement, compilation repair, and runtime feedback. Overall, our work reframes logging statement generation as a runtime-guided process that accounts for LLM-based downstream use, and advocates evaluating generated logging statements by their downstream utility rather than textual similarity.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; *Software maintenance tools*; • **Computing methodologies** → *Natural language processing*.

Keywords

logging statements generation, large language models, automated debugging

ACM Reference Format:
Xin Wang, Yang Feng, Xiaoqian Jiao, Yang Zhang, Zhenhao Li, and Zishuo Ding. 2026. ReLog: Execution-Aware Logging with Runtime Feedback for LLM-Oriented Debugging. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3832783.3837463

1 Introduction

Software developers insert logging statements into source code to expose useful information about program execution. During execution, these statements produce runtime logs that capture detailed system behaviors and states [23, 45], thereby supporting a wide range of software engineering tasks, such as debugging, performance analysis, and system monitoring [3, 4, 15, 20, 22, 29, 33, 41]. As illustrated in Figure 1, a typical logging statement consists of a severity level (e.g., info, debug), a static text template (e.g., "{} execution time: ms."), and dynamic variables (e.g., method name, execution time), which together record the runtime information about a program event for subsequent analysis.

Despite their importance, writing effective logging statements remains challenging for developers [6, 15, 22]. To alleviate this challenge, researchers have conducted a variety of studies aimed at improving logging quality and better supporting developers. Some investigate developers' logging preferences and practices [6, 7, 11,

* Equal contribution.
† Corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3837463

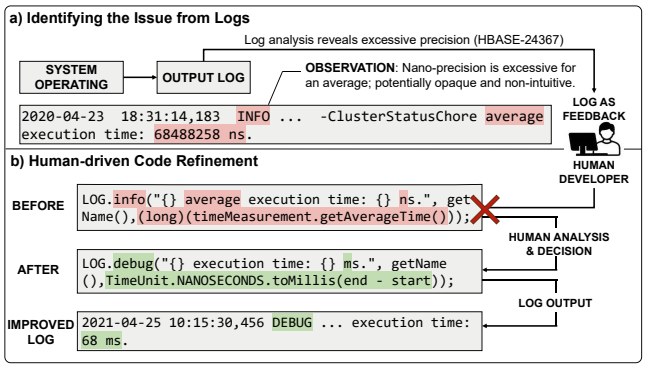


Figure 1: The iterative refinement process of a logging statement in Apache HBase. Developers initially recorded execution time in nanoseconds but later revised the code to millisecond precision and a lower severity level based on actual runtime observations.

12, 14, 25–27, 30, 36, 44], while others propose techniques for automatically suggesting or generating logging statements [9, 10, 16, 24, 26, 28, 45, 46]. These studies largely take a developer-centric perspective, focusing on what kinds of logging statements developers prefer and how to generate statements that resemble human-written ones. However, this perspective is becoming increasingly limited in the LLM era, where logging statements are consumed not only by human developers but also by LLMs for downstream tasks such as defect localization, failure analysis, and program repair [13, 17, 21, 34, 40]. Meanwhile, prior work [24, 35, 45] typically evaluates generated logging statements by comparing them against developer-written ones, implicitly assuming that the original statements constitute an adequate gold standard. This assumption may also be questionable in practice, because developer-written logging statements may themselves be suboptimal [6, 11, 25, 27, 36], and similarity to such statements does not necessarily reflect their practical utility for log-based downstream tasks. Therefore, we argue that automatic logging statement generation should move beyond merely imitating developer-written statements and instead aim to produce logging statements that are actually useful for downstream tasks, while also accounting for the needs of LLMs as emerging consumers of runtime logs.

In addition, existing approaches to automatic logging statement generation typically formulate the task as a static, single-pass generation problem, where the input is source code or other contextual information, such as abstract syntax trees, and the output is either a logging statement or a code snippet containing one. However, this formulation is inherently limited. In practice, logging statements are often refined iteratively based on the runtime logs they produce (cf. Section 2): developers inspect the generated runtime logs, identify deficiencies in the logging statement, and then revise its content, granularity, or severity level accordingly. For example, as illustrated in Figure 1, in issue HBASE-24367 [1], developers initially introduced a logging statement to record the average execution time of background chores in nanoseconds. However, the resulting runtime logs revealed that such fine-grained precision was neither intuitive for operators nor representative of the underlying performance characteristics. Consequently, the developers revised the logging statement to report execution time in milliseconds and adjusted the severity level to a more appropriate one. This refinement process highlights that effective logging often depends on runtime behaviors and execution states that cannot be fully inferred from static code context alone. As a result, static approaches may generate syntactically correct logging statements while still failing to capture the diagnostic information needed in practice.

To address the above limitations, we propose **ReLog**, a runtime feedback-driven framework for iterative logging statement generation. Instead of treating logging statement generation as a static single-pass prediction task, **ReLog** formulates it as an execution-aware refinement process. **ReLog** is intended for offline debugging workflows, such as CI/CD failure diagnosis and local reproduction, rather than repeatedly modifying logging code on production hot paths. Given a target program and a reproducible failure, **ReLog** first executes the original code to capture its execution outcome, and then generates initial logging statements based on both the code context and the observed runtime behavior. After inserting the generated statements, **ReLog** compiles and executes the instrumented program, using a repair loop to resolve any compilation errors introduced by the added logging statements. It then invokes an LLM-based module to assess whether the resulting runtime logs provide sufficient observability for downstream tasks. If the diagnostic evidence is still insufficient, **ReLog** employs a refiner module to update the logging statements based on the feedback, and repeats the process iteratively. In this way, **ReLog** goes beyond generating logging statements that merely resemble developer-written ones, and instead aims to produce logging statements whose resulting runtime logs are useful for LLM-based downstream tasks.

To evaluate the practical utility of generated logging statements, we construct two new downstream debugging datasets from Defects4J [19]. These datasets are designed to assess whether generated logging statements provide sufficient diagnostic information for downstream debugging under different levels of source-code availability. The first dataset, *direct debugging*, contains 311 instances across 16 projects, where the LLM-based debugging agent is given both the faulty source code and the generated runtime logs for defect detection and program repair. The second dataset, *indirect debugging*, contains 225 instances across 15 projects and simulates settings where the faulty source code is unavailable, requiring the agent to reason using only the generated runtime logs and caller context collected from the failing execution path.

Our evaluation shows that **ReLog** consistently outperforms all baselines in both direct and indirect debugging. In the setting of *direct debugging*, where source code is available, it achieves the highest bug detection F1 score of 0.520, exceeding the best baseline by 16.33%, and successfully repairs 97 defects. In the setting of *indirect debugging*, where debugging relies only on runtime logs and caller context, it still attains the best F1 score of 0.408, outperforming the top baseline by 16.57%. These benefits remain consistent when varying both the LLMs used inside **ReLog** and the downstream LLMs that consume the generated logs, indicating that the gains come mainly from the framework itself. Ablation studies further show that compilation repair, iterative refinement, and runtime feedback are essential. Finally, a qualitative case study demonstrates

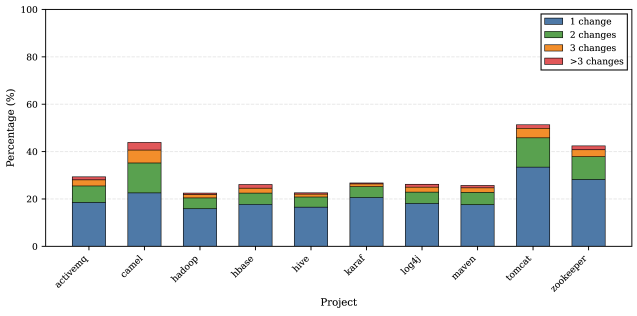


Figure 2: Distribution of change frequencies for logging statements across ten open source projects. The figure displays the percentage of statements revised one to three or more times.

how the runtime feedback loop helps **ReLog** progressively build informative evidence for exposing the root causes of defects.

We summarize the contributions of this paper as follows:

- We propose **ReLog**, a runtime feedback-driven framework for iterative logging statement generation. The replication package of this paper is available [2].
- We introduce a new evaluation methodology that assesses generated logging statements by their examining utility in downstream debugging tasks, rather than by computing similarity to developer-written statements.
- We construct two Defects4J-based datasets for direct and indirect debugging, enabling the evaluation of generated logging statements under different levels of source-code availability.

Paper Organization. The remainder of this paper is organized as follows. Section 2 presents a motivating study on iterative logging refinement. Section 3 introduces the design of **ReLog**. Section 4 presents the experimental setup. Section 5 reports the evaluation results. Section 6 reviews related work. Section 7 discusses the threats to validity, and Section 8 concludes the paper.

2 Motivating Study

To better understand how logging statements evolve in real world software development workflows, we conduct a motivating study on their change histories. Our goal is to examine whether developers refine logging statements after their initial insertion rather than treating them as fixed instrumentation, which would provide practical motivation for iterative logging statement generation and downstream-utility-oriented evaluation. Following prior studies on logging [24, 25, 36, 44, 45], we study ten Apache projects from diverse domains: Hadoop, HBase, Hive, Tomcat, ActiveMQ, Camel, Log4j, ZooKeeper, Maven, and Karaf. For each project, we traverse its version history and identify logging statements using common logging APIs, such as `logger.debug`, and `logger.info`. We then track each logging statement across commits and count how many times it is modified. This allows to characterize how frequently logging statements are revised in practice.

Figure 2 summarizes the distribution of modification frequencies for logging statements that were changed at least once. Across all ten projects, most modified logging statements are revised only once. However, a substantial fraction is modified two or three times, and some statements are revised more than three times. Beyond this distribution, we also observe that, when considering all identified logging statements in each project, more than 20% undergo at least one modification. In projects such as Tomcat, Camel, and ZooKeeper, this proportion even exceeds 40%.

These results indicate that logging statements are not always fixed after their initial insertion. Instead, developers do revise them in practice, sometimes multiple times, to better support evolving diagnostic needs. This observation suggests that treating logging statement generation as a purely static, single-pass task may not fully align with real development workflows.

It is worth noting that our measurement relies solely on committed changes recorded in version histories. In practice, developers may temporarily add, remove, or adjust logging statements during local debugging sessions before committing the final code. Such edits are typically not preserved in the repository history. Therefore, the modification frequencies reported in this study likely represent a conservative estimate, and the actual extent of logging refinement during development may be even greater [8, 37].

Overall, this motivating study provides empirical support for viewing logging statement generation as an iterative problem. It also motivates the need for approaches that account for refinement based on runtime observations, rather than relying solely on static one-pass generation.

3 Methodology

3.1 Overview

Figure 3 presents the overall architecture of **ReLog**, our runtime feedback-driven framework for iterative logging statement generation. Different from existing approaches that typically formulate logging statement generation as a static and single-pass task based on source code alone, **ReLog** treats it as an execution-aware refinement process, motivated by the observation that whether a logging statement is effective often becomes clear only after examining the runtime logs it produces during execution.

Given an input program, **ReLog** operates as a closed-loop system with four stages. It first generates initial logging statements using both source code context and observed execution outcomes, then repairs any compilation issues introduced by the newly-added logging statements, evaluates whether the resulting runtime logs provide sufficient information for downstream tasks, and iteratively refines the logging statements when necessary. Through this iterative loop of generation, execution, evaluation, and refinement, **ReLog** aims to produce logging statements whose resulting runtime logs are useful for downstream tasks in modern LLM-assisted software engineering workflows. We next describe these four stages in detail.

3.2 Framework Workflow

This subsection details the four stages of **ReLog**'s refinement loop, corresponding to the workflow illustrated in Figure 3.

Stage 1: Initial Logging Statements Generation. Existing approaches typically generate logging statements from source code or other static contextual information alone. **ReLog** extends this

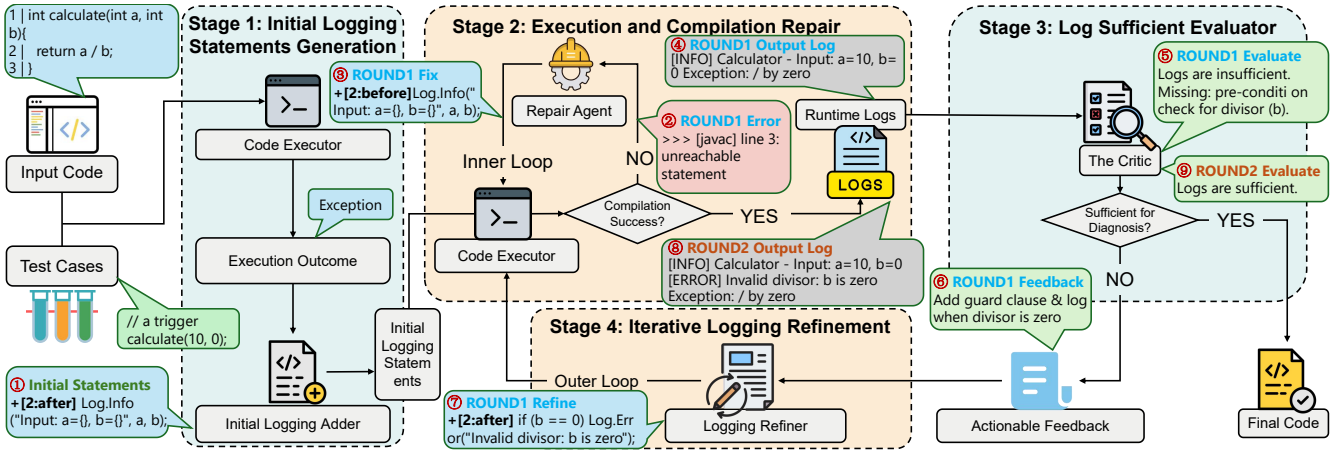


Figure 3: The overall architecture of ReLog with a two-round example.

formulation by additionally incorporating runtime execution signals when available. Specifically, **ReLog** first executes the input code snippet to obtain its initial execution outcome. If the program runs normally, the framework falls back to a source-code-based generation setting similar to prior work. If execution exposes useful diagnostic signals, such as an exception, an incorrect return value, or a timeout, **ReLog** leverages this information to narrow the search space and focus logging generation on potentially fault-relevant parts of the code.

ReLog then uses the captured execution outcome together with the input code to generate targeted initial logging statements. To prevent unintended modifications to the original logic, the framework prompts the language model to output a discrete list of logging statements paired with their specific insertion line numbers rather than regenerating the entire code snippet. The execution outcome serves as an early diagnostic signal that guides instrumentation toward code regions likely to contribute to the observed failure. For example, if a `NullPointerException` occurs at a specific line, **ReLog** can prioritize nearby statements and relevant variables when generating logging statements. In this way, Stage 1 subsumes static logging generation as a special case while providing a more informative starting point for later stages of compilation repair, log evaluation, and iterative refinement.

Stage 2: Compilation Repair. After the generated logging statements are inserted into the code, the resulting instrumented program may fail to compile. Such failures can be caused by unresolved variables, incompatible API usage, missing imports, or other inconsistencies introduced during logging insertion. A logging statement that resembles developer-written code is still of limited practical value if its insertion breaks compilation or prevents subsequent execution. Existing approaches to logging statement generation mainly focus on static generation quality, such as textual similarity to developer-written statements, while paying limited attention to whether the inserted logging statements remain compilable and executable in the target program. As we show later in our evaluation, prior approaches may generate uncompilable logging statements, which break the original program.

ReLog therefore includes a compilation repair stage that uses compiler feedback to fix errors induced by the inserted logging statements. When compilation fails, the framework collects the compiler error messages and uses them to revise the generated logging code, while keeping the original program logic unchanged. Since the original program is inherently compilable, any compilation failure is strictly caused by the newly added statements. To ensure the original logic remains intact, the repair module exclusively targets and modifies the previously generated list of logging statements based on the compiler feedback. The framework then reinserts this updated list into the pristine original code. This repair loop continues until the instrumented program compiles successfully or a predefined retry limit is reached. By ensuring the executability of the inserted logging statements, this stage enables subsequent runtime evaluation and iterative refinement.

Stage 3: Log Sufficiency Evaluation. After the instrumented program successfully compiles and executes, **ReLog** collects the resulting runtime logs and evaluates whether they provide sufficient information for downstream log-based tasks. This stage is motivated by a common practice in real-world software development: developers often inspect the produced runtime logs, judge the current logging statements, and further refine them when the observed logs are still inadequate. **ReLog** follows the same intuition, but extends this traditionally human-centered judgment process by introducing an LLM-based critic to assess whether the current logs are sufficient for downstream tasks. This design reflects the growing role of LLMs in modern software engineering workflows, as runtime logs are increasingly consumed not only by human developers but also by LLMs [13, 17, 21, 40].

Specifically, inspired by prior empirical studies on logging practices [3, 14, 22, 41, 42], the critic is guided by a structured evaluation rubric to assess the current runtime logs along three predefined dimensions. The first is **traceability**, which examines whether the logs clearly expose the execution path relevant to the observed behavior. The second is **state visibility**, which checks whether key variables and intermediate states are adequately recorded. The third is **causal linkage**, which evaluates whether the logs provide enough evidence to explain why the observed behavior occurred,

rather than merely showing that it occurred. Based on this structured assessment, the critic produces both a sufficiency judgment and actionable feedback for refinement. This rubric-based design is flexible and extensible: although **ReLog** currently adopts these three dimensions, developers can incorporate additional criteria or task-specific requirements when needed, allowing the critic to adapt to different downstream scenarios beyond a fixed evaluation template.

Operationally, the critic receives the original code with line numbers, the applied logging statements, the runtime logs, and the execution outcome. It is prompted to mark the logs as sufficient only when the evidence exposes the relevant execution path, records the state needed to reason about the failure, and connects the recorded state to the observed outcome. Otherwise, it identifies the missing evidence and emits concrete refinement suggestions. The generator, fixer, critic, and refiner are implemented as separate LLM roles with separate conversations, so the sufficiency judgment is not produced by the same conversation that inserted the current statements.

Finally, if the critic determines that the current logs are sufficient, the framework terminates successfully. Otherwise, it generates feedback to guide the next refinement step. For example, the critic may identify that an important variable is updated inside a loop but never logged, and suggest inserting a logging statement at a specific location. In this way, Stage 3 serves as the key bridge between runtime log observation and iterative logging statement refinement to ensure diagnostic quality.

Stage 4: Iterative Logging Statement Refinement. When the critic identifies observability gaps, **ReLog** enters the iterative refinement stage. A logging refiner module acts as an actor that updates the current logging statements based on the structured feedback produced in Stage 3.

Instead of regenerating logging statements from scratch, the refiner performs targeted modifications to address the identified deficiencies. Because the framework maintains the generated logging statements as a discrete list independent of the source code, the language model can directly manipulate this list. Guided by the diagnostic feedback, the refiner emits one of four logging-only actions: Add new statements, Modify the variables or message content of existing statements, Remove redundant statements, or Relocate statements to more informative execution points. These actions may add missing variable states, adjust logging position to better capture dynamic control flow, or remove redundant information that introduces noise. The refiner is explicitly constrained not to change the original program logic; the updated list is reinserted into the pristine original code before the next compilation and execution. This design allows **ReLog** to preserve useful existing instrumentation while incrementally improving the parts that limit downstream log utility.

Once the logging statements are updated, the revised code is fed back into Stage 2 for compilation repair and execution, and the resulting runtime logs are reevaluated in Stage 3. The loop continues until the logs are judged sufficient for downstream tasks or a predefined maximum number of iterations is reached (five in our experiments). In this way, **ReLog** progressively improves the diagnostic quality of the generated logging statements through a closed-loop refinement process.

4 Experiment Setup

4.1 Downstream Evaluation Task

Prior work [24, 35, 45] on logging statement generation is typically evaluated against developer-written statements using static matching-based metrics, such as position and level accuracy, and textual similarity. However, as discussed in the **Introduction**, this evaluation has limitations: developer-written logging statements may themselves be suboptimal, and static metrics do not directly capture whether the resulting runtime logs are useful for downstream tasks. To address this limitation, we propose to evaluate **ReLog** through downstream tasks to assess whether the generated logging statements provide useful diagnostic information for practical debugging scenarios.

Specifically, we choose debugging tasks as the evaluation target, since it is a primary application of runtime logs [4, 5, 13, 21, 34] and serves as a practical testbed for assessing whether the generated logging statements provide useful diagnostic information. In particular, we consider two settings that reflect different levels of source-code availability: *direct debugging*, where the faulty code is accessible, and *indirect debugging*, where diagnosis must rely on runtime logs and calling context alone.

4.1.1 Direct Debugging. In this setting, the debugging agent has access to both the faulty source code and the generated runtime logs. This setup aligns with common development scenarios, where engineers can inspect the code while using logs to understand the failure.

This task consists of two distinct subtasks: (i) **Defect Localization.** Given the code and the corresponding runtime logs, the agent determines whether a defect exists and produces a textual description of the identified issue. A prediction is considered a *True Positive* when the generated defect description matches the ground truth, and Precision, Recall, and F1 Score are used to evaluate localization effectiveness. (ii) **Program Repair.** If a defect is detected, the agent is prompted to repair the faulty code using the generated defect description together with the available runtime information. The final output is a candidate patch. A repair is considered successful if the generated patch is identical or semantically equivalent to the ground-truth fix.

4.1.2 Indirect Debugging. In this setting, the faulty method itself is inaccessible to the downstream debugging agent. The debugging process therefore relies on runtime logs and caller-side context observed on the failing execution path. This setup reflects operational environments where developers may not have direct access to the faulty source file, but can still inspect surrounding caller context or failure-triggering traces.

Accordingly, this setting focuses only on **defect localization**. Given the runtime logs produced by the instrumented program and the available calling context, the agent determines whether a defect exists and generates a textual description of the issue. This scenario evaluates whether the generated logs alone provide sufficient information to diagnose failures when the source code is hidden. Similarly, we use Precision, Recall, and F1 Score as the evaluation metric.

4.1.3 Debugging Agents. LLM-based agents are increasingly used to assist developers in software engineering tasks [18, 40]. As a result, it is increasingly important to provide runtime logs that are useful for LLM-based tasks. Following this trend, we implement lightweight LLM-based debugging agents to perform the above-mentioned tasks in our evaluation. For localization, the agent takes the available code context and runtime logs as input, and outputs a defect decision together with a textual description of the issue. For repair, the agent takes the faulty code and the generated defect description as input, and outputs a candidate patch. Our purpose is not to propose a state-of-the-art agent for defect localization or repair, but to examine whether the generated runtime logs provide useful information for LLM-based debugging.

To evaluate localization, two authors independently compare each predicted defect description with the ground-truth defect and mark it correct when it identifies the same faulty behavior and causal mechanism, rather than requiring exact wording. To evaluate repair, they mark a patch correct when it is textually identical to the ground-truth fix or implements the same behavior. Disagreements are resolved by a third author.

4.2 Dataset Construction

To support the downstream evaluation, we construct our datasets from the widely used Defects4J benchmark [19], as existing resources do not directly provide such settings. Based on different levels of source-code availability, we derive two datasets: one for direct debugging and the other for indirect debugging. Note that our goal is not to advance the state of the art on Defects4J itself, but to use Defects4J-based downstream debugging tasks as a practical testbed for evaluating whether generated logging statements provide useful diagnostic information.

Direct Debugging Dataset: Each sample is centered on a single faulty method. We retain reproducible Defects4J cases whose faulty method can be identified by comparing buggy and fixed versions, whose buggy and fixed method bodies are extractable, and whose triggering test reliably reproduces the failure. The resulting dataset contains 311 samples from 16 projects.

Indirect Debugging Dataset: For each sample, we additionally extract the direct caller methods that appear on the actual invocation path leading to the failure. We obtain these callers by instrumenting test executions and recording method invocation chains, while retaining only project source methods with resolvable bodies. The downstream agent cannot access the faulty method, but can access the extracted caller context. The resulting dataset contains 225 samples from 15 projects, each augmented with at least one caller method resolved at runtime.

4.3 Baselines

We evaluate our approach against several representative baselines that span diverse automated logging generation paradigms, ranging from traditional deep neural networks to modern LLMs.

- **SCLogger** [24] generates logging statements by leveraging static cross method contexts and variable type information to determine logging positions, severity levels, and message content.
- **UniLog** [39] relies on in context learning with demonstration examples to generate logging statements.
- **LANCE** [32] utilizes deep learning to predict the insertion position and generate a single textual logging statement.
- **LANCE2** [31] extends LANCE by combining deep learning and information retrieval to determine logging necessity and support multiple statement injections.
- **FastLog** [38] improves generation efficiency by predicting token level insertion positions to construct complete logging statements during the generation process.

5 Research Questions

In this section, we aim to answer the following research questions:

RQ1: How Well Does ReLog Support Downstream Debugging Tasks?

Motivation. Existing approaches to logging statement generation are typically static and evaluated using surface-level matching metrics. In contrast, **ReLog** introduces runtime feedback and iterative refinement. The key question is whether this design improves the practical usefulness of generated logging statements. Therefore, we investigate whether **ReLog** more effectively supports downstream debugging tasks, including defect localization and program repair.

Approach. We conduct our experiments using the direct debugging and indirect debugging datasets. For each defective code snippet, we apply **ReLog** and all selected baselines to generate logging statements. To ensure a fair comparison, **ReLog** and all baselines are based on DeepSeek-V3 because of its balance between cost and efficiency. We attempt to compile and execute the updated code to collect the generated runtime logs. We record any compilation failure as an unsuccessful attempt. Finally, we evaluate the successfully collected logs through downstream debugging tasks.

Results. **ReLog achieves the highest overall debugging performance in the direct debugging setting.** As shown in Table 1, **ReLog** attains the highest F1 score for defect localization at **0.520**. This performance surpasses UniLog with an F1 score of 0.447, GoStatic at 0.431, and the baseline without logging at 0.418. This strong performance stems from both a higher precision of 0.530 and a recall of 0.511. These metrics indicate that **ReLog** detects more true defects while simultaneously producing fewer incorrect detections. In terms of absolute numbers, **ReLog** correctly identifies 159 out of 311 defects. Meanwhile, the strongest baseline, UniLog, detects 130 defects. Furthermore, **ReLog** achieves the highest repair performance by successfully fixing **97 defects**, which accounts for 31.19% of the total cases. This result surpasses GoStatic with 78 repairs and UniLog with 63 repairs. Compared to earlier deep learning approaches such as LANCE with 38 repairs and FastLog with 34 repairs, **ReLog** increases the repair success rate by more than a factor of two. These results demonstrate that logging refinement guided by runtime feedback produces statements that effectively assist downstream debugging and repair tasks.

ReLog also consistently outperforms all baselines in the indirect debugging setting. When the source code is unavailable and debugging relies primarily on runtime logs, **ReLog** still achieves the highest localization performance with an F1 score of **0.408**. This result exceeds the strongest baseline, GoStatic, which achieves an F1 score of 0.350. In absolute terms, **ReLog** correctly

Table 1: Comparison with Baseline Logging Approaches on Direct and Indirect Debugging Tasks

Method	Direct Debugging							
	Compilation Failures	Detected Defects	True Positives	Precision	Recall	F1 Score	Successful Repairs	Avg. Logs
ReLog	**0**	**300**	**159**	**0.530**	**0.511**	**0.520**	**97**	5.52
ReLog $_{1\text{-}log}$	0	276	142	0.514	0.457	0.484	84	1.00
UniLog	30	270	130	0.481	0.418	0.447	63	0.97
GoStatic	18	288	129	0.448	0.415	0.431	78	7.63
Without Log	0	283	124	0.438	0.399	0.418	72	0
LANCE	74	225	77	0.342	0.248	0.288	38	0.58
FastLog	142	162	65	0.401	0.209	0.275	34	0.88
LANCE2	171	135	38	0.281	0.122	0.170	22	0.76
Method	Indirect Debugging							
	Compilation Failures	Detected Defects	True Positives	Precision	Recall	F1 Score	Avg. Logs per Caller	
ReLog	**2**	**142**	**75**	**0.528**	**0.333**	**0.408**	6.21	
GoStatic	78	112	59	0.527	0.262	0.350	9.76	
ReLog $_{1\text{-}log}$	29	100	57	0.570	0.253	0.350	1.00	
UniLog	12	64	32	0.500	0.142	0.221	0.94	
LANCE	56	52	21	0.404	0.093	0.151	0.47	
LANCE2	184	27	12	0.444	0.053	0.095	0.66	
FastLog	134	42	12	0.286	0.053	0.089	0.54	

detects 75 out of 225 defects. In comparison, GoStatic detects 59 defects and UniLog detects 32 defects. The performance gap becomes even larger when compared to deep learning approaches such as LANCE with an F1 score of 0.151 and LANCE2 with an F1 score of 0.095. These findings indicate that the iterative refinement process produces statements capturing informative runtime behaviors to enable reliable debugging even when the underlying source code remains inaccessible during the diagnosis.

ReLog maintains a consistently lower compilation failure rate than most baselines. In the direct debugging scenario, **ReLog** introduces zero compilation failures. Conversely, several baselines frequently generate uncompilable code. For example, FastLog produces 142 failures and LANCE2 produces 171 failures. A similar trend appears in the indirect debugging setting. Here, **ReLog** results in only two compilation failures, compared to 78 failures for GoStatic and 184 failures for LANCE2. This stability highlights how the integrated repair module successfully maintains executable logging code throughout the entire refinement process.

Generated logs provide additional value beyond source code alone. The direct debugging setting gives the downstream agent access to the faulty source code, so the "Without Log" baseline helps isolate the contribution of generated runtime logs. Removing generated logs reduces F1 from 0.520 to 0.418 and successful repairs from 97 to 72. This shows that **ReLog**'s logs provide diagnostic evidence beyond the code itself. The gap is even clearer in the indirect setting, where the agent cannot inspect the faulty method and must rely on logs plus caller context.

ReLog overcomes baseline limitations by generating sufficient diagnostic logging statements. Evaluation results show that deep learning baselines like FastLog, LANCE, and LANCE2 average fewer than one logging statement per method. This restriction stems from their fundamental task formulation, as these approaches are designed for a one shot generation paradigm. Although LANCE2 incorporates multiple generation, its objective remains focused on optimizing individual predictions instead of producing comprehensive diagnostic traces. Likewise, UniLog relies on in context learning with demonstrations derived from the same single insertion paradigm, leading to a comparable bias of 0.97 logs per method. These designs inherently restrict the capacity to capture complex execution states, since a single statement rarely exposes the necessary control flow and intermediate variables for debugging. In contrast, **ReLog** adopts an unconstrained paradigm guided by runtime feedback, enabling the generation and iterative refinement of multiple logging statements. Consequently, **ReLog** produces highly informative diagnostic traces, averaging 5.52 logs per method in the direct setting and 6.21 per caller in the indirect setting. Furthermore, **ReLog** remains more concise than static analysis baselines like GoStatic, which generate 7.63 and 9.76 logs respectively. These findings demonstrate that **ReLog** transcends the constraints of single statement generation, ensuring sufficient diagnostic observability without excessive verbosity.

ReLog remains effective under a one-log budget. Average log count is a descriptive overhead statistic rather than a direct measure of log sufficiency, since one statement may record multiple variables. To address logging-budget fairness, we additionally evaluate **ReLog** $_{1\text{-}log}$, a constrained variant that permits only one generated logging statement. In direct debugging, this variant achieves an F1 score of 0.484, higher than all non-**ReLog** baselines, while using the same average log budget as single-insertion methods. In indirect debugging, it reaches an F1 score of 0.350, matching GoStatic and exceeding UniLog, LANCE, LANCE2, and FastLog. These results show that **ReLog**'s gains are not solely explained by inserting more statements; adaptive multi-log refinement further improves performance when the debugging task requires richer observability.

RQ1 Summary: ReLog consistently outperforms existing baseline approaches across both direct and indirect debugging scenarios. By successfully improving defect localization and program repair rates, **ReLog** demonstrates strong robustness whether the source code is accessible or completely unavailable. These outcomes confirm that iterative refinement guided

by execution feedback produces high quality logging statements, providing benefits for downstream debugging tasks.

RQ2: How Does ReLog Perform Across Different Large Language Models?

Motivation. Our framework utilizes LLMs for the generation, evaluation, and refinement of logging statements. However, these models vary substantially in their parameter size, training data, reasoning capacity, and ability to follow instructions. It remains unclear whether the effectiveness of **ReLog** depends on a specific model or whether the proposed refinement mechanism based on runtime feedback generalizes across different models. Evaluating **ReLog** with multiple models allows us to assess its robustness and helps determine whether the observed performance benefits stem from the iterative refinement design rather than the inherent capabilities of a particular model.

Approach. We evaluate **ReLog** using several representative LLMs with varying scales and architectures across both datasets. To ensure consistent evaluation, we maintain identical experimental settings for all models, including the limits on refinement iterations and compilation repair strategies. First, we vary the upstream LLM used by the **ReLog** generator, fixer, critic, and refiner while keeping the downstream debugging agent fixed. Second, to test whether generated logs remain useful to different log consumers, we reuse each method's generated logs and evaluate downstream localization with GPT-5-mini, Qwen3-Coder-30B, and GLM-4.7. We assess whether the iterative refinement mechanism consistently enhances downstream debugging performance regardless of the underlying generation or consumption model.

Results. **ReLog demonstrates strong robustness across different language models in the direct debugging setting.** As shown in Table 2, all evaluated models achieve comparable performance when integrated into **ReLog**. This indicates that the effectiveness of the framework is not strictly tied to a specific model. Among the tested LLMs, GPT-5-mini achieves the highest defect localization performance with an F1 score of **0.559**. GLM-4.7 follows closely with an F1 score of 0.551. DeepSeek-V3 and Qwen3-Coder-30B achieve scores of 0.520 and 0.518 respectively. These results show that the localization performance remains consistently high across different models, with the differences falling within a narrow margin. Furthermore, all models achieve strong repair performance. GPT-5-mini successfully repairs 90 bugs, and GLM-4.7 repairs 92 bugs. This consistent success demonstrates that the iterative refinement mechanism effectively guides the debugging process regardless of the underlying language model.

ReLog maintains consistent performance across models in the indirect debugging setting. GPT-5-mini again achieves the highest localization performance with an F1 score of **0.430**. DeepSeek-V3 follows with a score of 0.408, while GLM-4.7 and Qwen3-Coder-30B achieve 0.382 and 0.378 respectively. In terms of absolute localization counts, GPT-5-mini correctly identifies 80 defects out of 225 total cases, while DeepSeek-V3 detects 75 defects. Although the models differ in their architecture and scale, the overall performance gap remains moderate. This observation suggests that the debugging effectiveness primarily originates from the framework design rather than the specific capabilities of the underlying model.

The performance variations across models are smaller than the overall benefits provided by the ReLog framework. When compared against the baseline methods reported in RQ1, all tested LLM variants within **ReLog** consistently outperform existing logging generation approaches. For instance, even the model with the lowest performance in the direct debugging setting, Qwen3-Coder-30B with an F1 score of 0.518, still surpasses the strongest baseline shown in Table 1. This finding indicates that the refinement mechanism guided by runtime feedback plays a more critical role in assisting debugging tasks than the selection of LLMs.

Different language models exhibit varying logging generation behaviors while maintaining stable debugging performance. For example, Qwen3-Coder-30B generates the highest number of logs on average, producing 7.50 statements per method in the direct debugging scenario. In contrast, GLM-4.7 generates fewer logs with an average of 4.41 statements. Despite these distinct differences in logging quantity, all models achieve similar levels of debugging effectiveness. This outcome suggests that the iterative refinement process successfully adapts the generated logs to the specific requirements of the debugging task. It ensures that the necessary runtime information is captured regardless of the specific generation style of the chosen model.

ReLog-generated logs remain useful when consumed by different downstream LLMs. Table 3 reports F1 scores when the downstream log consumer is changed while the generated logs are reused. Across all four downstream LLMs and both debugging settings, **ReLog** achieves the best F1 score among all logging methods. With DeepSeek-V3 as the downstream consumer, **ReLog** reaches 0.520 F1 in direct debugging and 0.408 F1 in indirect debugging. With GPT-5-mini, **ReLog** reaches 0.678 and 0.459. With the locally hosted Qwen3-Coder-30B, **ReLog** still obtains the highest F1 scores of 0.421 and 0.369. With GLM-4.7, **ReLog** again ranks first with 0.552 and 0.462. This cross-consumer analysis reduces the risk that **ReLog** merely tailors logs to a single downstream agent and supports the claim that runtime-refined logs encode generally useful diagnostic evidence.

RQ2 Summary: ReLog consistently achieves strong debugging performance across different upstream generation models and downstream log-consuming models. Although performance varies across models, **ReLog** remains the top-performing logging method in every evaluated cross-consumer setting. This consistency confirms that the effectiveness of **ReLog** primarily stems from its iterative refinement mechanism guided by runtime feedback, rather than a strict reliance on any particular language model.

RQ3: What Is the Impact of Key Components in ReLog?

Motivation. **ReLog** integrates three core mechanisms: a compilation repair module, iterative refinement, and runtime feedback. To validate the overall architecture of our framework, we isolate and

Table 2: Impact of Different LLMs on ReLog Performance

Model	Direct Debugging							
	Compilation Failures	Detected Defects	True Positives	Precision	Recall	F1 Score	Successful Repairs	Avg. Logs
DeepSeek-V3	0	300	159	0.530	0.511	0.520	97	5.52
Qwen3-Coder-30B	3	299	158	0.528	0.508	0.518	84	7.50
GLM-4.7	0	299	168	0.562	0.540	0.551	92	4.41
GPT-5-mini	2	297	170	0.572	0.547	**0.559**	90	6.43
Model	Indirect Debugging							
	Compilation Failures	Detected Defects	True Positives	Precision	Recall	F1 Score	Successful Repairs	Avg. Logs per Caller
DeepSeek-V3	2	142	75	0.528	0.333	0.408	–	6.21
Qwen3-Coder-30B	12	113	64	0.566	0.284	0.378	–	5.60
GLM-4.7	5	116	65	0.560	0.289	0.382	–	3.86
GPT-5-mini	8	148	80	0.541	0.356	**0.430**	–	8.06

Table 3: F1 Scores with Different Downstream LLM Log Consumers under Direct and Indirect Debugging

Downstream LLM	Direct Debugging					
	ReLog	UniLog	GoStatic	LANCE	FastLog	LANCE2
DeepSeek-V3	**0.520**	0.447	0.431	0.288	0.275	0.170
GPT-5-mini	**0.678**	0.593	0.570	0.462	0.349	0.302
Qwen3-Coder-30B	**0.421**	0.305	0.369	0.218	0.179	0.145
GLM-4.7	**0.552**	0.488	0.454	0.331	0.291	0.188
Downstream LLM	Indirect Debugging					
	ReLog	UniLog	GoStatic	LANCE	FastLog	LANCE2
DeepSeek-V3	**0.408**	0.221	0.350	0.151	0.089	0.095
GPT-5-mini	**0.459**	0.363	0.366	0.273	0.213	0.150
Qwen3-Coder-30B	**0.369**	0.147	0.221	0.109	0.084	0.033
GLM-4.7	**0.462**	0.253	0.370	0.174	0.110	0.083

evaluate the individual contribution of each element. Specifically, we investigate whether the observed performance benefits originate primarily from successfully resolving compilation errors, from multi-round source-based refinement, or from continuously enhancing the logging statements through actual execution results. This ablation analysis justifies our specific design choices and confirms the necessity of these mechanisms to achieve optimal performance.

Approach. We evaluate the individual contribution of each core component by testing three ablated variants of **ReLog**. We maintain identical experimental settings across all evaluations. The first variant completely disables the compilation repair module. Consequently, the generation process terminates immediately upon encountering any compilation error. The second variant disables runtime feedback while preserving the same multi-round source-based refinement budget, so the model can revise logs but cannot use actual runtime logs as refinement evidence. The third variant removes the iterative refinement loop, limiting the framework to a single static generation pass. Comparing these results against the full framework quantifies the specific impact of each component.

Results. **Compilation repair, runtime feedback, and iterative refinement each contribute measurably to ReLog.** As shown in Table 4, disabling any of these mechanisms leads to a noticeable degradation in debugging performance across both evaluation settings. In the direct debugging scenario, the full version of **ReLog** achieves an F1 score of **0.520**. It correctly detects 159 bugs and successfully repairs 97 defects. When we remove the compilation fixer, the F1 score drops to 0.439, and the number of successful repairs decreases to 75. Disabling runtime feedback reduces the F1 score to 0.461 and successful repairs to 76. Removing the refinement mechanism reduces the F1 score further to 0.388 and brings the successful repairs down to 81. These consistent drops indicate that all three mechanisms play important roles in assisting downstream debugging tasks.

The compilation repair module is essential for maintaining stable executable instrumentation. Without the fixer, compilation failures increase dramatically. In the direct debugging setting, the number of compilation failures rises from zero to **94**. In the indirect debugging setting, this number increases from 2 to **106**. This large influx of compilation errors directly reduces the number of analyzable executions. Consequently, it leads to a lower recall rate. For instance, the recall drops to 0.366 compared to 0.511 in the full direct debugging setup. These results confirm that automatically repairing compilation errors introduced during logging insertion is critical. This repair ensures that the system can reliably collect actual runtime feedback.

The iterative refinement mechanism enhances diagnostic utility and acts synergistically with compilation repair. Removing the refinement loop reduces the framework to a traditional static logging generator. Under the direct debugging setting, the detection F1 score drops from **0.520** to 0.388, and the total count of correctly detected bugs decreases from 159 to 118. The indirect debugging scenario shows a similar trend, with the F1 score decreasing from 0.408 to 0.364. These results confirm that refinement driven by runtime feedback incrementally captures execution information relevant to the underlying failures. Ultimately, these ablation findings highlight a complementary system design. While the repair module ensures execution stability by preventing instrumentation failures, the refinement mechanism leverages the resulting data to improve log informativeness. This synergy enables **ReLog** to consistently generate logs that are both executable and diagnostically useful.

Runtime feedback provides gains beyond multi-round source-based refinement. When runtime feedback is disabled but refinement remains available, the direct debugging F1 score decreases from 0.520 to 0.461, and successful repairs decrease from 97 to 76. In the indirect setting, F1 decreases from 0.408 to 0.368. This variant also generates more logs in the indirect setting, averaging 12.50 logs per caller, but still performs worse than full **ReLog**. These results indicate that the benefit does not come merely from

Table 4: Ablation Study of ReLog Components

Variant	Direct Debugging							
	Compilation Failures	Detected Defects	True Positives	Precision	Recall	F1 Score	Successful Repairs	Avg. Logs
ReLog	0	300	159	0.530	0.511	**0.520**	97	5.52
ReLog w/o Fixer	94	208	114	0.548	0.366	0.439	75	4.79
ReLog w/o Runtime Feedback	17	287	138	0.481	0.443	0.461	76	1.79
ReLog w/o Refine	1	297	118	0.397	0.379	0.388	81	4.50
Variant	Indirect Debugging							
	Compilation Failures	Detected Defects	True Positives	Precision	Recall	F1 Score	Avg. Logs per Caller	
ReLog	2	142	75	0.528	0.333	**0.408**	6.21	
ReLog w/o Fixer	106	102	58	0.569	0.258	0.355	6.37	
ReLog w/o Runtime Feedback	29	122	62	0.554	0.276	0.368	12.50	
ReLog w/o Refine	14	116	62	0.534	0.276	0.364	5.41	

allowing more rounds or more statements. Runtime execution evidence helps the critic identify concrete observability gaps and helps the refiner add logs that explain the actual failing behavior.

RQ3 Summary: The debugging effectiveness of **ReLog** stems from the synergy between compilation repair, runtime feedback, and iterative refinement. The repair module ensures execution stability, runtime feedback exposes concrete observability gaps, and the refinement mechanism uses this evidence to progressively enhance logging quality.

Case Study

To demonstrate how **ReLog** enhances downstream debugging in practice, we conduct a qualitative case study on a representative defect from our benchmark. We compare the logging statements and the resulting execution traces produced by **ReLog** against baseline approaches, specifically GoStatic and LANCE. This detailed examination reveals how our iterative generation paradigm successfully provides actionable evidence for automated fault localization compared to traditional static methods.

As illustrated in Figure 4, evaluating the original code without any logging statements causes the debugging agent to miss the defect entirely. Static generation approaches struggle to improve this outcome. For instance, LANCE introduces a compilation error during log insertion, which prevents execution and directly leads to a detection failure. While GoStatic successfully compiles the code and generates execution traces, these logs only record the method entry and the final unexpected return value. Because they fail to expose why the conditional logic bypasses the target branch, the agent detects an anomaly but incorrectly explains the root cause.

In contrast, **ReLog** systematically constructs a complete chain of diagnostic evidence through compilation repair and iterative refinement. Initially, the framework places a log immediately after a return statement. The compilation repair module detects this unreachable code error and automatically repositions the statement before the return, ensuring that the instrumented program can execute. The first collected trace shows that the conditional check fails and the method returns a null value. Determining that this lacks sufficient context, the log sufficiency evaluator suggests logging the exact values of the class constants for a direct comparison. The refiner then augments the logging code to print both the input raw class name and the target constant values. The subsequent execution clearly exposes the hidden type mismatch. Armed with this precise structural evidence, the downstream agent correctly identifies the missing alias check and accurately explains the defect.

6 Related Work

6.1 Logging Statements Generation

Many existing research explores the automatic generation of logging statements to assist developers in collecting runtime information software systems. These approaches typically address three key challenge in logging design. They determine the optimal locations for logging, the appropriate severity levels, and the specific information to record in the messages. Early studies focused on predicting logging locations by training machine learning models on source code features [26, 46]. Subsequent research extended this direction. Researchers applied deep learning techniques to recommend suitable logging levels based on the surrounding code context [16, 28]. More recent approaches leverage neural sequence generation and LLMs. These advanced methods generate complete logging statements that include both the textual message content and the relevant code variables [9, 10, 23, 24, 31, 32, 38, 39, 43–45].

Despite recent advances, existing techniques generate logging statements through a static single pass over source code while ignoring program execution behavior. Furthermore, these methods evaluate generation quality using text similarity and position prediction rather than measuring debugging utility. In contrast, our work introduces an iterative framework that refines logging statements using runtime feedback and evaluates its effectiveness through downstream debugging tasks.

6.2 Logging Statements Practice

Some researchers investigates logging practices and defects related to logging in production software systems. Prior studies analyze logging usage patterns by mining version histories and issue reports. For instance, researchers characterize common logging practices and recurring challenges. These challenges include deciding what information to record and structuring the textual messages [6]. Other studies examine specific logging issues in detail. They identify temporal inconsistencies between source code and logs [11]. They also detect duplicated logging statements [27] and highlight readability problems in logging messages [25]. Together, these empirical studies provide valuable insights into the design of logging

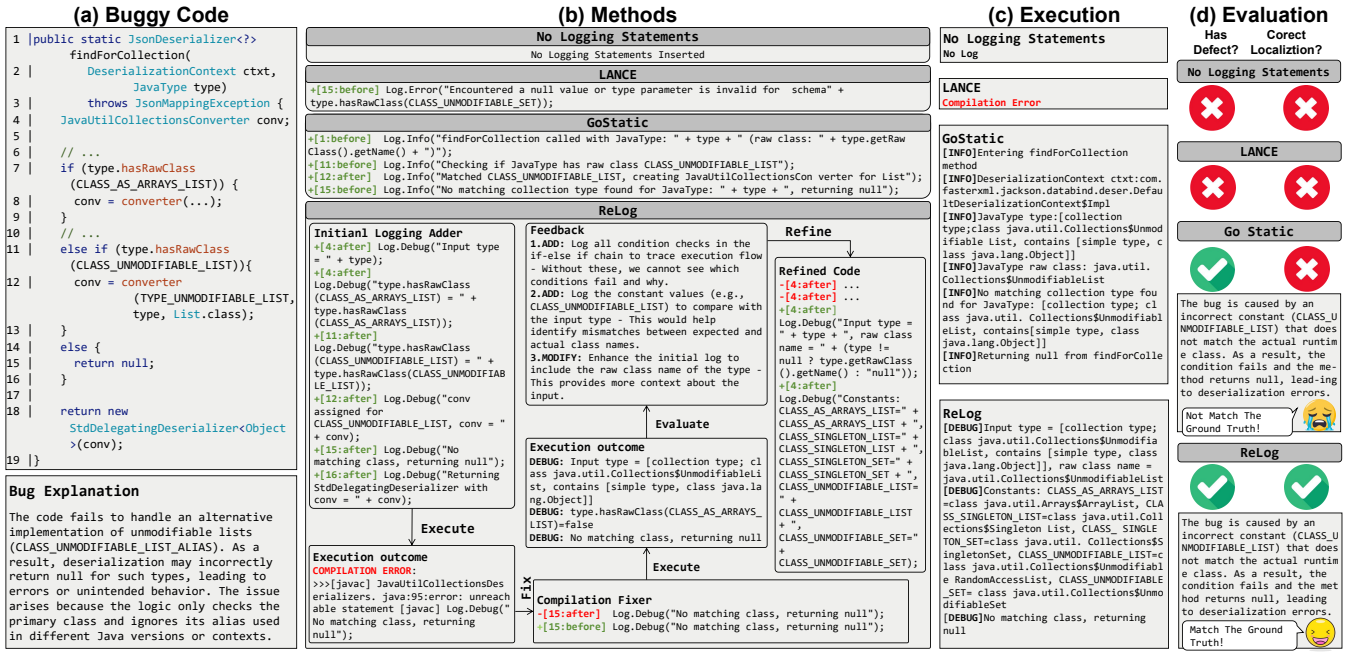


Figure 4: A comparative case study of logging generation approaches on a representative defect. The figure illustrates the inserted logging statements, the collected runtime log, and the final defect localization outcomes for ReLog and the evaluated static baselines.

statements and expose common pitfalls in practical software development and maintenance workflows.

While prior research primarily assists human developers through guidelines and pattern mining, the increasing reliance on LLMs demands logging strategies optimized for automated maintenance. Consequently, our work shifts the focus from human readability to generation tailored specifically for downstream software engineering tasks such as automated debugging.

7 Threats to Validity

Internal Validity.

Threats to internal validity concern factors within our experiment that could influence the evaluation results. A primary threat is the dependency on structurally valid input code. Because the iterative refinement mechanism of **ReLog** relies on dynamic execution, compilation errors in languages like Java block runtime signal extraction and could skew the performance assessment. To mitigate this, we evaluated our approach on a curated Java dataset where the input codebases are inherently executable, thereby isolating the effectiveness of our logging generation from basic syntax resolution.

External Validity.

Threats to external validity concern the generalizability of our findings. A key threat is the applicability of **ReLog** to diverse execution environments. Since iterative refinement introduces runtime overhead, its direct use in latency-sensitive production systems is limited. We mitigate this by scoping **ReLog** to offline debugging workflows, such as CI/CD failure diagnosis and local reproduction, where developers can invoke the framework after failures occur. Another threat is that our benchmark is based on Java defects from Defects4J. Although it provides reproducible failures and ground-truth fixes, future work should validate **ReLog** on more languages, projects, and bug types. Finally, we focus on automated debugging as the downstream task, as defect diagnosis is a critical use case for logging and offers a rigorous benchmark for log quality. Extending **ReLog** to anomaly detection, performance monitoring, readability assessment, or runtime-overhead optimization remains future work and can be supported by adapting the critic rubric.

8 Conclusion

In this paper, we introduce **ReLog** to overcome the limitations of static logging statement generation. Instead of relying solely on static code structures, **ReLog** iteratively refines logging statements using LLMs guided by execution feedback. Extensive evaluations on direct and indirect debugging tasks show that this dynamic approach consistently outperforms existing baselines, generalizes across upstream and downstream LLMs, and benefits from runtime feedback beyond multi-round source-based refinement. By resolving compilation errors and optimizing log content through runtime observations, **ReLog** generates informative execution traces even when the faulty source code is inaccessible to the downstream agent. Overall, this work establishes a new paradigm for automated logging generation, showing that prioritizing diagnostic utility over static text matching better bridges generated logs and downstream automated debugging.

9 Data Availability

To facilitate reproducibility and future research, the complete replication package for **ReLog** is available [2]. Researchers can access all necessary scripts and instructions to reproduce the findings presented in this paper.

10 Acknowledgments

This research was supported by the National Natural Science Foundation of China (No. 62502409).

References

[1] [n. d.]. HBASE-24367. https://issues.apache.org/jira/browse/HBASE-24367.
[2] Anonymous. 2026. ReLog Replication Package. doi:10.6084/m9.figshare.31866106
[3] Titus Barik, Robert DeLine, Steven Drucker, and Danyel Fisher. 2016. The Bones of the System: A Case Study of Logging and Telemetry at Microsoft. In *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*. doi:10.1145/2889160.2889231
[4] An Ran Chen. 2019. An Empirical Study on Leveraging Logs for Debugging Production Failures. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 126–128. doi:10.1109/ICSE-Companion.2019.00055
[5] An Ran Chen, Tse-Hsun Chen, and Shaowei Wang. 2022. Pathidea: Improving Information Retrieval-Based Bug Localization by Re-Constructing Execution Paths Using Logs. *IEEE Transactions on Software Engineering* 48, 8 (Aug. 2022), 2905–2919. doi:10.1109/TSE.2021.3071473
[6] Boyuan Chen and Zhen Ming Jiang. 2017. Characterizing and Detecting Anti-Patterns in the Logging Code. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. doi:10.1109/ICSE.2017.15
[7] Boyuan Chen and Zhen Ming Jiang. 2019. Extracting and studying the Logging-Code-Issue- Introducing changes in Java-based large-scale open source software systems. *Empirical Softw. Engg.* (2019). doi:10.1007/s10664-019-09690-0
[8] Yi-Hung Chou, Yiyang Min, April Yi Wang, and James A. Jones. 2025. Learning from Mistakes: Understanding Ad-hoc Logs through Analyzing Accidental Commits. doi:10.48550/arXiv.2501.09892
[9] Zishuo Ding, Heng Li, and Weiyi Shang. 2022. LoGenText: Automatically Generating Logging Texts Using Neural Machine Translation. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. doi:10.1109/SANER53432.2022.00051
[10] Zishuo Ding, Yiming Tang, Xiaoyu Cheng, Heng Li, and Weiyi Shang. 2023. LoGenText-Plus: Improving Neural Machine Translation Based Logging Texts Generation with Syntactic Templates. *ACM Trans. Softw. Eng. Methodol.* (2023). doi:10.1145/3624740
[11] Zishuo Ding, Yiming Tang, Yang Li, Heng Li, and Weiyi Shang. 2023. On the Temporal Relations between Logging and Code. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. doi:10.1109/ICSE48619.2023.00079
[12] Patrick Loic Foalem, Foutse Khomh, and Heng Li. 2023. Studying Logging Practice in Machine Learning-based Applications. doi:10.48550/arXiv.2301.04234
[13] Mirazul Haque, Petr Babkin, Farima Farmahinifarahani, and Manuela Veloso. 2025. Towards Effectively Leveraging Execution Traces for Program Repair with Code LLMs. doi:10.18653/v1/2025.knowledgenlp-1.17
[14] Mehran Hassani, Weiyi Shang, Emad Shihab, and Nikolaos Tsantalis. 2018. Studying and detecting log-related issues. *Empirical Softw. Engg.* (2018). doi:10.1007/s10664-018-9603-z
[15] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. 2021. A Survey on Automated Log Analysis for Reliability Engineering. *ACM Comput. Surv.* (2021). doi:10.1145/3460345
[16] Yi Wen Heng, Zeyang Ma, Zhenhao Li, Dong Jae Kim, and Tse-Hsun Chen. 2025. Benchmarking Open-Source Large Language Models for Log Level Suggestion. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. 314–325. doi:10.1109/ICST62969.2025.10988921
[17] Cheng Ji and Huaiying Luo. 2025. Leveraging Large Language Model for Intelligent Log Processing and Autonomous Debugging in Cloud AI Platforms. doi:10.48550/arXiv.2506.17900
[18] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? doi:10.48550/arXiv.2310.06770
[19] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA 2014)*. Association for Computing Machinery, New York, NY, USA, 437–440. doi:10.1145/2610384.2628055
[20] Jinhan Kim, Valeriy Savchenko, Kihyuck Shin, Konstantin Sorokin, Hyunseok Jeon, Georgiy Pankratenko, Sergey Markov, and Chul-Joo Kim. 2020. Automatic abnormal log detection by analyzing log history for providing debugging insight. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '20)*. doi:10.1145/3377813.3381371
[21] Youngjoon Kim, Sunguk Shin, Hyoungshick Kim, and Jiwon Yoon. 2025. Logs in, Patches out: Automated Vulnerability Repair via Tree-of-Thought LLM Analysis. In *Proceedings of the 34th USENIX Conference on Security Symposium*. Number 227. USENIX Association, USA, 4401–4419.
[22] Heng Li, Weiyi Shang, Bram Adams, Mohammed Sayagh, and Ahmed E. Hassan. 2021. A Qualitative Study of the Benefits and Costs of Logging From Developers' Perspectives. *IEEE Transactions on Software Engineering* (2021). doi:10.1109/TSE.2020.2970422
[23] Yichen Li, Yintong Huo, Zhihan Jiang, Renyi Zhong, Pinjia He, Yuxin Su, Lionel C Briand, and Michael R Lyu. 2024. Exploring the effectiveness of llms in automated logging statement generation: An empirical study. *IEEE Transactions on Software Engineering* (2024). doi:10.1109/TSE.2024.3475375
[24] Yichen Li, Yintong Huo, Renyi Zhong, Zhihan Jiang, Jinyang Liu, Junjie Huang, Jiazhen Gu, Pinjia He, and Michael R. Lyu. 2024. Go Static: Contextualized Logging Statement Generation. *Proc. ACM Softw. Eng.* (2024). doi:10.1145/3643754
[25] Zhenhao Li, An Ran Chen, Xing Hu, Xin Xia, Tse-Hsun Chen, and Weiyi Shang. 2023. Are They All Good? Studying Practitioners' Expectations on the Readability of Log Messages. doi:10.1109/ASE56229.2023.00136
[26] Zhenhao Li, Tse-Hsun Chen, and Weiyi Shang. 2020. Where Shall We Log? Studying and Suggesting Logging Locations in Code Blocks. In *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. doi:10.1145/3324884.3416636
[27] Zhenhao Li, Tse-Hsun Chen, Jinqiu Yang, and Weiyi Shang. 2019. DLFinder: Characterizing and Detecting Duplicate Logging Code Smells. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. doi:10.1109/ICSE.2019.00032
[28] Zhenhao Li, Heng Li, Tse-Hsun Chen, and Weiyi Shang. 2021. DeepLV: Suggesting Log Levels Using Ordinal Based Neural Networks. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. doi:10.1109/ICSE43902.2021.00131
[29] Zhenhao Li, Chuan Luo, Tse-Hsun Chen, Weiyi Shang, Shilin He, Qingwei Lin, and Dongmei Zhang. 2023. Did we miss something important? studying and exploring variable-aware log abstraction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 830–842. doi:10.1109/ICSE48619.2023.00078
[30] Zhongxin Liu, Xin Xia, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. 2021. Which Variables Should I Log? *IEEE Transactions on Software Engineering* 47, 9 (Sept. 2021), 2012–2031. doi:10.1109/TSE.2019.2941943
[31] Antonio Mastropaolo, Valentina Ferrari, Luca Pascarella, and Gabriele Bavota. 2023. Log Statements Generation via Deep Learning: Widening the Support Provided to Developers. doi:10.48550/arXiv.2311.04587
[32] Antonio Mastropaolo, Luca Pascarella, and Gabriele Bavota. 2022. Using Deep Learning to Generate Complete Log Statements. doi:10.48550/arXiv.2201.04837
[33] Fredrik Milani and Fabrizio M. Maggi. 2018. A Comparative Evaluation of Log-Based Process Performance Analysis Techniques. In *Business Information Systems* (2018). doi:10.48550/arXiv.1804.03965
[34] Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2024. NExT: Teaching Large Language Models to Reason about Code Execution. doi:10.48550/arXiv.2404.14662
[35] Boyin Tan, Junjielong Xu, Zhouruixing Zhu, and Pinjia He. 2025. AL-Bench: A Benchmark for Automatic Logging. doi:10.48550/arXiv.2502.03160
[36] Xin Wang, Zhenhao Li, and Zishuo Ding. 2025. Defects4Log: Benchmarking LLMs for Logging Code Defect Detection and Reasoning. doi:10.1109/ASE63991.2025.00161
[37] Fengcai Wen, Csaba Nagy, Michele Lanza, and Gabriele Bavota. 2020. An Empirical Study of Quick Remedy Commits. In *Proceedings of the 28th International Conference on Program Comprehension (ICPC '20)*. Association for Computing Machinery, New York, NY, USA, 60–71. doi:10.1145/3387904.3389266
[38] Xiaoyuan Xie, Zhipeng Cai, Songqiang Chen, and Jifeng Xuan. 2024. FastLog: An End-to-End Method to Efficiently Generate and Insert Logging Statements. doi:10.48550/arXiv.2311.02862
[39] Junjielong Xu, Ziang Cui, Yuan Zhao, Xu Zhang, Shilin He, Pinjia He, Liqun Li, Yu Kang, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2024. UniLog: Automatic Logging via LLM and In-Context Learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/3597503.3623326
[40] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=M4qNIzQYpd
[41] Ding Yuan, Haohui Mai, Weiwei Xiong, Lin Tan, Yuanyuan Zhou, and Shankar Pasupathy. 2010. SherLog: error diagnosis by connecting clues from run-time logs. In *Proceedings of the fifteenth International Conference on Architectural support*

for programming languages and operating systems. doi:10.1145/1736020.1736038

[42] Ding Yuan, Soyeon Park, and Yuanyuan Zhou. 2012. Characterizing logging practices in open-source software. In *2012 34th International Conference on Software Engineering (ICSE)*. doi:10.1109/ICSE.2012.6227202

[43] Renyi Zhong, Yintong Huo, Wenwei Gu, Yichen Li, and Michael R. Lyu. 2025. End-to-End Automated Logging via Multi-Agent Framework. doi:10.48550/arXiv.2511.18528

[44] Renyi Zhong, Yichen Li, Jinxi Kuang, Wenwei Gu, Yintong Huo, and Michael R. Lyu. 2025. LogUpdater: Automated Detection and Repair of Specific Defects in Logging Statements. *ACM Trans. Softw. Eng. Methodol.* (2025). doi:10.1145/3731754

[45] Renyi Zhong, Yichen Li, Guangba Yu, Wenwei Gu, Jinxi Kuang, Yintong Huo, and Michael R Lyu. 2025. Beyond LLMs: An Exploration of Small Open-source Language Models in Logging Statement Generation. *arXiv preprint arXiv:2505.16590* (2025). doi:10.48550/arXiv.2505.16590

[46] Jieming Zhu, Pinjia He, Qiang Fu, Hongyu Zhang, Michael R. Lyu, and Dongmei Zhang. 2015. Learning to Log: Helping Developers Make Informed Logging Decisions. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. doi:10.1109/ICSE.2015.60